



결제 수단 및 PG 결제

결제 수단 및 PG 결제

구현 범위

Premium Lifetime 단건 결제를 위해 **일반 카드(토스페이먼츠)** 와 **카카오페이**를 제공하고, PG 승인 결과를 결제 이력과 Premium 권한에 연결합니다.

1. 결제 화면 흐름

0 / 2

결제 진행

Premium 상세 분석 이용권 결제 수단을 선택합니다.

일반 카드

카드 정보를 입력하고 결제를 완료합니다.

일반 카드 결제

카카오페이

카카오페이 결제창에서 인증 후 결제를 완료합니다.

카카오페이로 결제

상품 정보

주문 번호

결제 시 자동 생성

결제 구분

Premium

상품명

Premium Lifetime

결제 금액

9,900원

화면 설계 자료 기준: 결제 수단 선택 후 PG 결제 단계로 진입합니다. 실제 구현에서는 일반 카드와 카카오페이를 제공합니다.

단계	사용자 흐름	서버 처리
0 / 2	일반 카드 또는 카카오페이 선택	활성 상품과 Premium 보유 여부 확인
1 / 2	PG 결제창에서 인증·결제 진행	주문 번호와 중복 방지 토큰으로 결제 요청 식별

결제 수단 및 PG 결제

1

단계	사용자 흐름	서버 처리
2 / 2	결제 완료 화면 확인	PAID 처리, PG 거래 식별자 저장, Premium Lifetime 권한 활성화

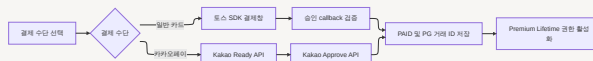
2. 결제 수단별 처리

일반 카드 - Toss Payments

- 결제 페이지에는 **Client Key**만 전달하고, Secret Key는 서버 설정 객체에서만 관리합니다.
- SDK 결제 완료 callback에서 주문 소유자, 주문 번호, 결제 금액을 다시 검증한 뒤 승인 API를 호출합니다.
- 동일 주문의 재요청은 기존 **PENDING** 주문을 재사용하고, 이미 승인된 주문은 idempotent하게 처리합니다.

카카오페이

- Ready API로 결제창 URL과 **tid**를 발급받아 카카오페이 결제창으로 이동합니다.
- tid**, 주문 번호, 사용자 ID, 상품 코드, idempotency token을 세션에 보관합니다.
- 승인 callback에서 결제 요청 사용자 일치 여부를 검증한 뒤 결제 이력과 Premium 권한을 생성합니다.



3. 핵심 코드 - Toss 승인 검증

`TossPaymentService.approve()`는 callback 값을 그대로 신뢰하지 않고, 저장된 주문과 대조한 뒤 승인합니다.

```
DummyPayment payment = loadOwnedTossPaymentForUpdate(userId, orderId);
if (PAYMENT_STATUS_PAID.equals(payment.getPaymentStatus()))
{
    return existingPaidResult(payment, userId, paymentKey);
}
if (!PAYMENT_STATUS_PENDING.equals(payment.getPaymentStatus
```

```

    )) {
        throw new BusinessException(ErrorCode.COMMON_IDEMPOTENCY_CONFLICT);
    }

    int expectedAmount = amount(payment.getAmount());
    if (callbackAmount != expectedAmount) {
        throw new BusinessException(
            ErrorCode.COMMON_VALIDATION_FAILED,
            "결제 금액이 주문 정보와 일치하지 않습니다."
        );
    }

    TossPaymentApproveResponse approved = confirm(paymentKey, orderId, expectedAmount);
    validateApprovedResponse(approved, paymentKey, orderId, expectedAmount);
    dummyPaymentMapper.markPaid(payment.getPaymentId(), PG_PROVIDER, approved.paymentKey());
    PremiumGrant grant = resolveGrant(userId, payment.getPaymentId());

```

- 사용자 소유 주문만 처리합니다.
- callback 금액과 저장된 결제 금액이 다르면 승인하지 않습니다.
- 승인 후에만 **PAID** 상태와 Premium grant를 생성합니다.

4. 핵심 코드 - 카카오페이 Ready 요청

`KakaoPayService.ready()` 는 Premium 활성 사용자 결제를 차단하고, PG 결제 요청에 필요한 식별 정보를 세션에 보관합니다.

```

if (premiumGrantService.hasActiveGrant(userId)) {
    throw new BusinessException(
        ErrorCode.COMMON_VALIDATION_FAILED,
        "이미 Premium 권한이 활성화되어 있습니다."
    );
}

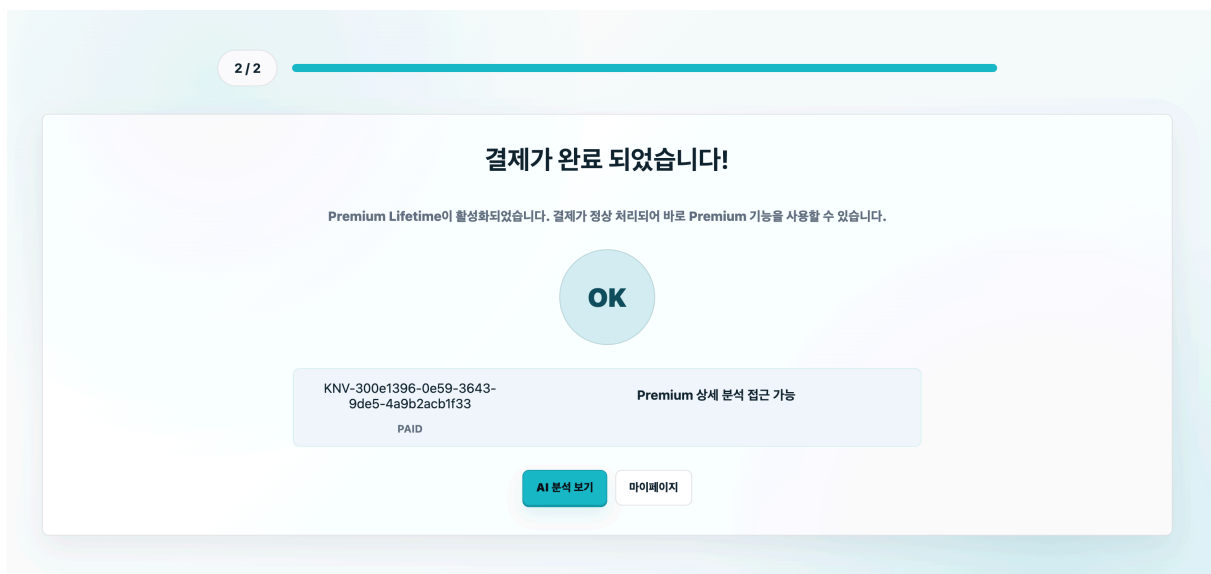
```

```
String orderNo = dummyPaymentService.buildOrderNo(
    userId, form.getIdempotencyToken()
);
KakaoPayReadyResponse response = postReady(userId, product,
orderNo);
String redirectUrl = response.redirectUrl();

httpSession.setAttribute(SESSION_KEY, new ReadySession(
    response.tid(), orderNo, userId,
    product.getProductCode(), form.getIdempotencyToken()
));
return redirectUrl;
```

- 중복 방지 token으로 주문 번호를 생성합니다.
- `tid` 와 요청 사용자 정보를 세션에 보관해 callback 위변조를 방지합니다.
- Secret Key는 서버에서만 API 요청 Header에 사용하며 화면에 노출하지 않습니다.

5. 결제 완료와 권한 연결



화면 설계 자료 기준: 결제 완료는 `2 / 2` 단계에서 안내합니다. 실제 구현은 승인 성공 시 결제 이력과 Premium 권한을 함께 갱신합니다.

- 성공: `PAID` 상태, PG 거래 식별자, Premium Lifetime grant 생성
- 취소/실패: `CANCELED` 또는 `FAILED` 상태만 기록하고 Premium 권한은 활성화하지 않음

- 이미 Premium인 사용자: 결제 수단 진입과 PG ready 요청을 모두 차단
- 결제 완료 후: AI 분석 화면과 설정의 결제 정보에서 이용 상태 및 결제 이력 확인 가능